Dataset 1: students_advanced.csv

1. Load the dataset and display the first 10 rows

```
import pandas as pd
df = pd.read_csv('/students_advanced.csv')   # 'NA' and 'nan' are read as NaN automa
df.head(10)
```

	Name	Age	Department	Attendance	Marks	Grade
0	Student_18	20.0	CS	100%	39	D
1	Student_15	19.0	Physics	70%	69	C
2	Student_19	20.0	Biology	88%	75	B
3	Student_04	21.0	Biology	82%	75	B
4	Student_24	20.0	Math	100%	67	C
5	Student_11	18.0	Physics	88%	42	D
6	Student_21	21.0	Math	70%	41	D
7	Student_22	18.0	Math	NaN	46	D
8	Student_05	19.0	Physics	NaN	63	C
9	Student_03	22.0	Physics	82%	94	A

Next steps: Generate code with df | New interactive sheet

2. Explore the dataset

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27 entries, 0 to 26
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Name        27 non-null     object
 1   Age         25 non-null     float64
 2   Department  27 non-null     object
 3   Attendance  22 non-null     object
 4   Marks       27 non-null     int64
 5   Grade       27 non-null     object
dtypes: float64(1), int64(1), object(4)
memory usage: 1.4+ KB
```

```
df.describe(include='all')
```

	Name	Age	Department	Attendance	Marks	Grade
count	27	25.000000	27	22	27.000000	27
unique	25	NaN	4	5	NaN	4
top	Student_04	NaN	CS	88%	NaN	D
freq	2	NaN	8	7	NaN	9
mean	NaN	20.000000	NaN	NaN	64.444444	NaN
std	NaN	1.354006	NaN	NaN	19.415167	NaN
min	NaN	18.000000	NaN	NaN	35.000000	NaN
25%	NaN	19.000000	NaN	NaN	45.500000	NaN
50%	NaN	20.000000	NaN	NaN	67.000000	NaN
75%	NaN	21.000000	NaN	NaN	75.000000	NaN
max	NaN	22.000000	NaN	NaN	99.000000	NaN

```
numerical_cols   = df.select_dtypes(include='number').columns.tolist()
categorical_cols = df.select_dtypes(exclude='number').columns.tolist()
print('Numerical  :', numerical_cols)
print('Categorical:', categorical_cols)
print('\nNote: Attendance is stored as text (e.g. "88%"), so it is categorical for
print('\nUnique Department:', df['Department'].unique())
print('Unique Grade     :', df['Grade'].unique())
```

```
Numerical  : ['Age', 'Marks']
Categorical: ['Name', 'Department', 'Attendance', 'Grade']

Note: Attendance is stored as text (e.g. "88%"), so it is categorical for now; we co

Unique Department: ['CS' 'Physics' 'Biology' 'Math']
Unique Grade     : ['D' 'C' 'B' 'A']
```

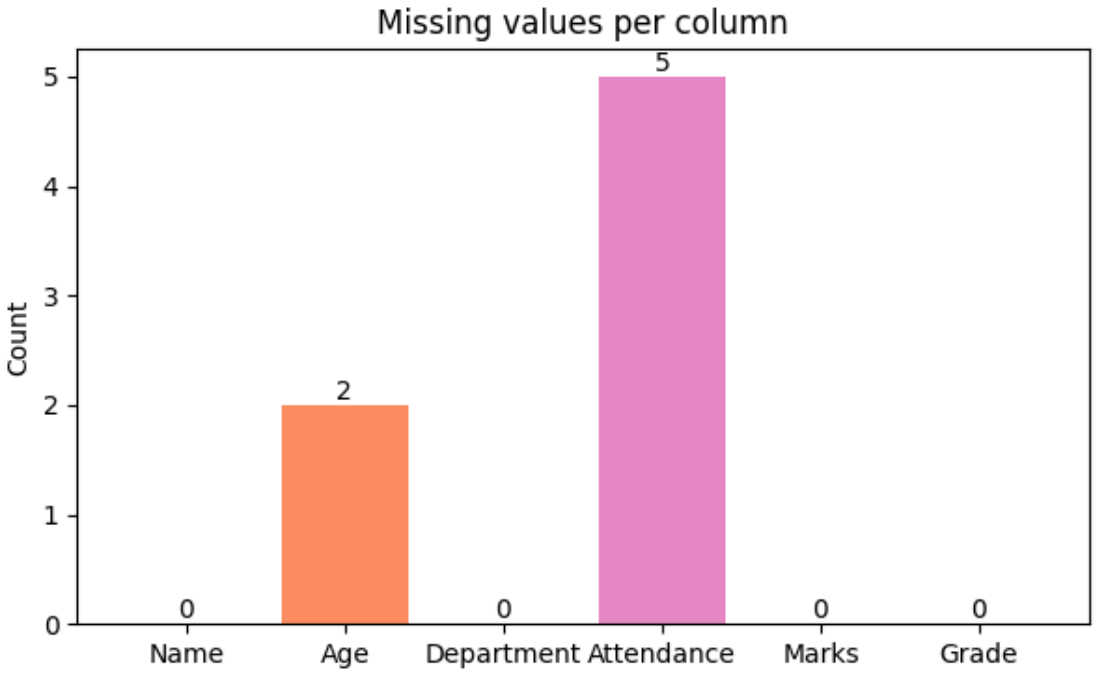
3. Identify and visualize missing values

```
missing = df.isnull().sum()
print(missing)
```

```
Name          0
Age           2
Department    0
Attendance    5
Marks         0
Grade         0
dtype: int64
```

```
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(7, 4))
colors = sns.color_palette('Set2', len(missing))
bars = plt.bar(missing.index, missing.values, color=colors)
plt.bar_label(bars)
```

```
plt.title('Missing values per column')
plt.ylabel('Count')
plt.show()
```



4. Handle missing values

Attendance must be converted to numeric before its median can be taken, so that conversion is done here first (it is also step 5).

```
# Attendance: '88%' -> 88.0 (the NaN values stay NaN)
df['Attendance'] = pd.to_numeric(df['Attendance'].astype(str).str.replace('%', '',

# Age -> mean rounded to 1 decimal
df['Age'] = df['Age'].fillna(round(df['Age'].mean(), 1))

# Grade -> mode
df['Grade'] = df['Grade'].fillna(df['Grade'].mode()[0])

# Attendance -> median
df['Attendance'] = df['Attendance'].fillna(df['Attendance'].median())

print(df.isnull().sum())
```

```
Name          0
Age           0
Department    0
Attendance    0
Marks         0
Grade         0
dtype: int64
```

5. Convert data types and standardize values

```
for col in ['Age', 'Marks', 'Attendance']:
    df[col] = pd.to_numeric(df[col], errors='coerce')
df.dtypes
```

	0
Name	object
Age	float64
Department	object
Attendance	float64
Marks	int64
Grade	object

dtype: object

6. Detect and remove duplicates

```
print('Shape before:', df.shape)
dups = df.duplicated().sum()
print('Total duplicates:', dups)
df = df.drop_duplicates().reset_index(drop=True)
print('Rows removed    :', dups)
print('Shape after     :', df.shape)
```

```
Shape before: (27, 6)
Total duplicates: 2
Rows removed    : 2
Shape after     : (25, 6)
```

7. Derived column: Result

```
import numpy as np
df['Result'] = np.where(df['Marks'] >= 50, 'Pass', 'Fail')
pass_pct = (df['Result'] == 'Pass').mean() * 100
print(f'Overall Pass Percentage: {pass_pct:.2f}%')
df.head()
```

```
Overall Pass Percentage: 64.00%
```

	Name	Age	Department	Attendance	Marks	Grade	Result
0	Student_18	20.0	CS	100.0	39	D	Fail
1	Student_15	19.0	Physics	70.0	69	C	Pass
2	Student_19	20.0	Biology	88.0	75	B	Pass
3	Student_04	21.0	Biology	82.0	75	B	Pass
4	Student_24	20.0	Math	100.0	67	C	Pass

Next steps: Generate code with df | New interactive sheet

8. Department-wise performance analysis

```
dept = (df.groupby('Department')['Marks']
          .agg(Average='mean', Minimum='min', Maximum='max')
          .sort_values('Average', ascending=False)
          .round(2))
dept
```

Department	Average	Minimum	Maximum
Biology	69.00	49	79
Physics	66.57	42	97
CS	58.57	35	99
Math	57.50	41	69

Next steps: Generate code with dept | New interactive sheet

```
print('Departments with average above 70:')
above70 = dept[dept['Average'] > 70]
print(above70 if not above70.empty else 'None - no department has an average above
```

```
Departments with average above 70:
None - no department has an average above 70.
```

9. Advanced filtering and insights

```
high = df[(df['Attendance'] >= 85) & (df['Marks'] >= 75)]
high
```

	Name	Age	Department	Attendance	Marks	Grade	Result
2	Student_19	20.0	Biology	88.0	75	B	Pass
14	Student_08	22.0	CS	88.0	99	A	Pass

Next steps: Generate code with high | New interactive sheet

```
print('High performers per department:')
print(high.groupby('Department').size())
print('\nAverage Marks     :', round(high['Marks'].mean(), 2))
print('Average Attendance:', round(high['Attendance'].mean(), 2))
```

```
High performers per department:
Department
Biology    1
CS         1
dtype: int64
```

```
Average Marks     : 87.0
Average Attendance: 88.0
```

10. Save the cleaned dataset

```
df.to_csv('cleaned_students_advanced.csv', index=False)
print('Saved cleaned_students_advanced.csv', df.shape)
```

```
Saved cleaned_students_advanced.csv (25, 7)
```

11. Visualizations

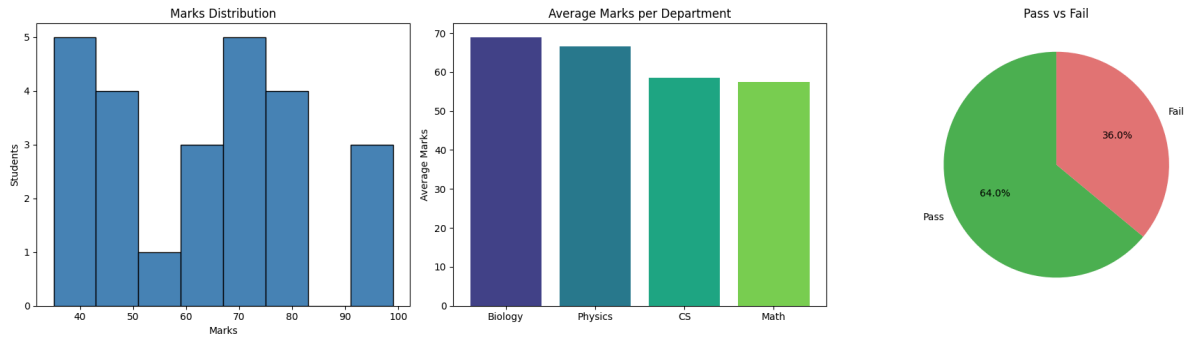
```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].hist(df['Marks'], bins=8, color='steelblue', edgecolor='black')
axes[0].set_title('Marks Distribution'); axes[0].set_xlabel('Marks'); axes[0].set_y

avg = df.groupby('Department')['Marks'].mean().sort_values(ascending=False)
axes[1].bar(avg.index, avg.values, color=sns.color_palette('viridis', len(avg)))
axes[1].set_title('Average Marks per Department'); axes[1].set_ylabel('Average Mark

counts = df['Result'].value_counts()
axes[2].pie(counts.values, labels=counts.index, autopct='%1.1f%%', colors=['#4caf50
axes[2].set_title('Pass vs Fail')

plt.tight_layout(); plt.show()
```



12. Bonus: correlation matrix and heatmap

```
corr = df[['Age', 'Marks', 'Attendance']].corr()
print(corr.round(3))
plt.figure(figsize=(6, 5))
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1, fmt='.2f')
plt.title('Correlation Matrix - Students')
plt.show()
```

	Age	Marks	Attendance
Age	1.000	0.123	0.129
Marks	0.123	1.000	-0.224
Attendance	0.129	-0.224	1.000

